

PHP

Introduzione a PHP

Cos'è PHP, dove viene usato e come iniziare a costruire pagine web dinamiche.

Cos'è PHP?

PHP è un **linguaggio di programmazione per il web**. Serve soprattutto a creare pagine che cambiano in base ai dati, all'utente o a quello che succede sul sito.

Una pagina HTML normale è come un volantino stampato: tutti vedono la stessa cosa. Una pagina PHP, invece, può preparare contenuti diversi prima di mandarli al browser.

```
<?php
$nome = "Luca";
echo "Ciao, $nome!";
```

Questo codice crea un messaggio usando un valore salvato nella variabile `$nome`.

Lato server e lato client

Quando apri un sito, il tuo browser riceve HTML, CSS e JavaScript. Questi lavorano sul tuo computer o telefono: per questo si dice **lato client**.

PHP lavora prima, sul computer che ospita il sito: il **server**. Il server esegue PHP, produce HTML e poi manda il risultato al browser.

```
<?php
echo "<h1>Benvenuto</h1>";
```

Il browser non vede il codice PHP. Vede solo l'HTML prodotto.

Dove si usa PHP

PHP è usato per:

- pagine web dinamiche
- form di contatto
- aree riservate con login
- siti con database
- blog, negozi online e pannelli di amministrazione

Molti progetti famosi, incluso WordPress, usano PHP.

Cosa imparerai in questa guida

Partiremo dalle basi: installazione, sintassi, variabili e tipi di dato. Poi vedremo condizioni, cicli, funzioni e array.

Dopo le fondamenta passeremo al web: HTML, form, sessioni, cookie e upload di file. Infine vedremo database, oggetti, gestione degli errori e strumenti come Composer.

L'obiettivo non è imparare tutto a memoria. È capire abbastanza bene da leggere codice PHP semplice, modificarlo con sicurezza e costruire piccoli programmi web.

Cosa devi sapere prima di iniziare

Puoi iniziare anche se non hai mai scritto PHP. Ti aiuterà conoscere almeno l'idea di una pagina HTML, perché PHP viene spesso usato per produrre HTML dinamico.

Non devi conoscere database, oggetti o strumenti professionali. Li incontrerai più avanti, quando le basi saranno già più familiari.

Come studiare questa guida

PHP si capisce meglio scrivendo codice piccolo e osservando cosa succede. Per ogni pagina prova a fare tre cose:

1. copia l'esempio principale in un file `.php`
2. eseguilo con il terminale o con il server locale
3. cambia un valore e guarda come cambia il risultato

Questo passaggio è importante. Leggere codice aiuta, ma modificarlo ti fa capire davvero quali parti contano.

Un esempio molto piccolo

PHP può prendere un dato e usarlo per costruire una risposta.

```
<?php
$prodotto = "quaderno";
$prezzo = 3.50;

echo "Il prodotto $prodotto costa $prezzo euro.";
```

Output:

```
Il prodotto quaderno costa 3.5 euro.
```

In poche righe hai già tre idee importanti: una variabile, un numero e un testo prodotto dal programma.

Percorso consigliato

Se parti da zero, segui le sezioni in questo ordine:

1. installazione e primo programma
2. sintassi, variabili, tipi di dato e operatori
3. condizioni e cicli
4. funzioni e scope
5. array, stringhe e date
6. PHP dentro pagine HTML e form
7. sessioni, cookie e upload
8. database con PDO e query preparate
9. classi, oggetti, errori e strumenti

Le ultime sezioni sono più vicine al lavoro su progetti reali. Non avere fretta di arrivarci: se variabili, condizioni, cicli e funzioni sono chiari, tutto il resto diventa meno misterioso.

Sintassi di base

Le regole essenziali per scrivere istruzioni PHP leggibili e corrette.

Il tag di apertura

Il codice PHP inizia con:

```
<?php
```

Questo tag avvisa PHP che le righe successive contengono istruzioni da eseguire.

```
<?php  
echo "Ciao";
```

Nei file che contengono solo PHP, di solito non si mette il tag di chiusura `?>`. Evitarlo riduce piccoli errori causati da spazi o righe vuote alla fine del file.

Il punto e virgola

In PHP molte istruzioni finiscono con `;`.

```
<?php  
$nome = "Marta";  
echo $nome;
```

Il punto e virgola dice: questa istruzione è finita. Se lo dimentichi, PHP spesso mostra un errore di sintassi.

Blocchi di codice

Le parentesi graffe `{ }` raccolgono più istruzioni dentro un blocco.

```
<?php
$eta = 20;

if ($eta >= 18) {
    echo "Puoi entrare";
}
```

Qui il messaggio viene mostrato solo se la condizione è vera.

PHP dentro HTML

PHP può vivere dentro una pagina HTML:

```
<h1>Profilo</h1>

<?php
$nome = "Luca";
echo "<p>Ciao, $nome</p>";
?>
```

HTML descrive la struttura della pagina. PHP prepara parti dinamiche.

Spazi e leggibilità

PHP non richiede spazi perfetti, ma il codice leggibile aiuta molto.

```
<?php
$prezzo = 10;
$ sconto = 2;
$totale = $prezzo - $sconto;

echo $totale;
```

Una riga vuota tra gruppi di istruzioni rende più chiaro cosa sta succedendo.

Commenti, istruzioni e blocchi insieme

Un piccolo file PHP può contenere tutti questi elementi:

```
<?php
// Dati di partenza
$nome = "Sara";
$eta = 21;

if ($eta >= 18) {
    echo "Ciao $nome, puoi continuare.";
} else {
    echo "Ciao $nome, questa area non è disponibile.";
}
```

La prima parte prepara i dati. La seconda parte controlla una condizione. Il blocco tra `{` e `}` contiene le istruzioni da eseguire se la condizione è vera.

Errore comune: confondere HTML e PHP

HTML non usa il punto e virgola e non esegue calcoli. PHP invece esegue istruzioni.

```
<p>Questo è HTML</p>

<?php
echo "Questo testo viene prodotto da PHP";
?>
```

Quando una pagina contiene entrambi, chiediti sempre: questa riga descrive la pagina o deve essere eseguita come istruzione?

Riepilogo

Per scrivere PHP corretto all'inizio ricorda quattro cose: apri il codice con `<?php`, chiudi le istruzioni con `;`, usa le graffe per i blocchi e tieni il codice leggibile con spazi e nomi chiari.

Commenti

Come usare i commenti per spiegare il codice PHP senza eseguirlo.

Cosa sono i commenti

Un commento è testo scritto per chi legge il codice. PHP lo ignora e non lo esegue.

I commenti servono a spiegare una scelta, ricordare un dettaglio o rendere più chiaro un passaggio non immediato.

Commenti su una riga

Usa `//` per scrivere un commento breve.

```
<?php
// Salviamo il nome dell'utente
$nome = "Giulia";

echo $nome;
```

PHP esegue `$nome = "Giulia";` ed `echo $nome;`, ma ignora la riga che inizia con `//`.

Commenti su più righe

Usa `/*` e `*/` quando il commento occupa più righe.

```
<?php
/*
Questo esempio calcola il prezzo finale
partendo da prezzo iniziale e sconto.
*/
$prezzo = 30;
$sconto = 5;

echo $prezzo - $sconto;
```

Quando usare i commenti

Un buon commento spiega il perché, non ripete l'ovvio.

```
<?php
// Applichiamo lo sconto solo agli ordini sopra i 50 euro
if ($totale > 50) {
    $totale = $totale - 10;
}
```

Questo commento è utile perché spiega la regola.

Quando evitarli

Questo commento non aiuta:

```
<?php
// Stampa il nome
echo $nome;
```

Il codice è già chiaro. Se devi commentare ogni riga, forse puoi scegliere nomi migliori o dividere il codice in pezzi più piccoli.

Commentare una scelta

Un commento diventa prezioso quando spiega una regola che non si vede subito dal codice.

```
<?php
$totale = 62;

// La spedizione è gratuita dagli ordini sopra i 50 euro
if ($totale > 50) {
    $costoSpedizione = 0;
} else {
    $costoSpedizione = 4.90;
}
```


Qui il commento non ripete `if ($totale > 50)` . Spiega la regola del negozio.

Usare i commenti mentre studi

Quando impari, puoi usare i commenti per dividere un esercizio in passaggi.

```
<?php
// 1. Salvo i dati iniziali
$prezzo = 20;
$sconto = 5;

// 2. Calcolo il totale
$totale = $prezzo - $sconto;

// 3. Mostro il risultato
echo $totale;
```

Questo aiuta a leggere il programma come una piccola storia. Con il tempo userai meno commenti, perché i nomi delle variabili e delle funzioni faranno già parte del lavoro.

Prova tu

Prendi un esercizio breve e aggiungi tre commenti: dati iniziali, calcolo, risultato. Poi rileggilo e cancella i commenti che ripetono soltanto quello che il codice dice già.

Tipi di dato

Numeri, stringhe, booleani, null e altri valori fondamentali in PHP.

Cosa significa tipo di dato

Un tipo di dato dice che genere di valore stai usando. Un numero, un testo e un vero/falso non si comportano nello stesso modo.

PHP capisce il tipo dal valore che assegna.

Numeri interi e decimali

Gli interi sono numeri senza virgola.

```
<?php  
$eta = 30;
```

I decimali si scrivono con il punto.

```
<?php  
$prezzo = 12.50;
```

Stringhe

Una stringa è testo. Si scrive tra virgolette.

```
<?php  
$nome = "Luca";  
$messaggio = 'Ciao';
```

Le virgolette doppie permettono di inserire variabili dentro il testo.

```
<?php  
$nome = "Luca";  
echo "Ciao $nome";
```

Booleani

Un booleano può essere solo `true` o `false`.

```
<?php
$maggiorenne = true;
$pagato = false;
```

Li userai spesso nelle condizioni.

Null

`null` significa: qui non c'è nessun valore.

```
<?php
$telefono = null;
```

Non vuol dire zero e non vuol dire stringa vuota. Vuol dire proprio assenza di valore.

Array

Un array raccoglie più valori in una sola variabile.

```
<?php
$colori = ["rosso", "verde", "blu"];
```

Controllare il tipo

Per osservare un valore puoi usare `var_dump`.

```
<?php
$eta = 30;
var_dump($eta);
```

Output possibile:

```
int(30)
```

`int` indica un numero intero.

Lo stesso valore puo avere tipi diversi

Guarda questi due valori:

```
<?php
$numero = 25;
$testo = "25";

var_dump($numero);
var_dump($testo);
```

Il primo e un numero intero. Il secondo e testo, anche se contiene cifre. Questa differenza conta quando fai confronti, calcoli o validazioni.

Conversioni semplici

A volte un valore arriva come stringa, per esempio da un form. Se vuoi trattarlo come numero, puoi convertirlo.

```
<?php
$etaDalForm = "32";
$eta = (int) $etaDalForm;

var_dump($eta);
```

`(int)` chiede a PHP di trasformare il valore in un intero.

Errore comune: confondere null e stringa vuota

```
<?php
$telefono = null;
$indirizzo = "";
```

`$telefono` non ha valore. `$indirizzo` ha un valore, ma è un testo vuoto. Nei controlli questa differenza può essere importante.

Riepilogo

Prima di usare un valore chiediti che cosa rappresenta: numero, testo, vero/falso, assenza di valore o raccolta di dati. Capire il tipo rende il codice più prevedibile.

Il primo programma PHP

Come creare ed eseguire un primo file PHP partendo da zero.

Creare il file

Apri la cartella degli esercizi e crea un file chiamato `ciao.php`.

Scrivi questo codice:

```
<?php
echo "Ciao, PHP!";
```

La riga `<?php` dice: da qui inizia codice PHP. La parola `echo` mostra un testo.

Eseguire il file dal terminale

Apri il terminale nella stessa cartella del file e scrivi:

```
php ciao.php
```

Output:

```
Ciao, PHP!
```

PHP legge il file ed esegue le istruzioni dall'alto verso il basso.

Aggiungere una seconda istruzione

Modifica il file così:

```
<?php
echo "Ciao, PHP!\n";
echo "Questo e il mio primo programma.";
```

`\n` crea una nuova riga nel terminale.

Output:

```
Ciao, PHP!
Questo e il mio primo programma.
```

Usare il server locale integrato

PHP può anche avviare un piccolo server locale. Serve quando vuoi vedere una pagina nel browser.

Crea `index.php` :

```
<?php
echo "<h1>Ciao dal browser</h1>";
```

Poi esegui:

```
php -S localhost:8000
```

Apri `http://localhost:8000` nel browser. Vedrai il titolo prodotto da PHP.

Nota: il server locale serve per studiare e provare. Per pubblicare un sito online servono impostazioni diverse.

Capire cosa hai appena fatto

Hai usato PHP in due modi diversi.

Nel primo caso hai eseguito un file dal terminale. Questo è utile per esercizi piccoli, prove veloci e script che non devono mostrare una pagina web.

Nel secondo caso hai avviato un server locale. Il browser ha chiesto una pagina a `localhost:8000`, PHP ha eseguito `index.php` e ha mandato al browser il risultato.

`localhost` significa "questo computer". Non è un sito pubblico: è un indirizzo speciale che usi per provare il progetto sulla tua macchina.

Errore comune: aprire il file direttamente

Se apri `index.php` facendo doppio clic sul file, il browser potrebbe mostrarti il codice o non eseguirlo correttamente. Per eseguire PHP serve passare da PHP stesso, con il comando:

```
php -S localhost:8000
```

Poi apri l'indirizzo nel browser.

Prova tu

Modifica il messaggio dentro `echo`, salva il file e ricarica la pagina. Se il testo cambia, hai completato il ciclo base: scrivi, salvi, esegui, osservi il risultato.

Installazione e ambiente

Come installare PHP, scegliere un editor e verificare che l'ambiente funzioni.

Cosa ti serve

Per scrivere PHP ti servono tre cose:

1. PHP installato sul computer
2. un editor di testo per scrivere codice
3. un terminale per eseguire i comandi

L'editor può essere Visual Studio Code, PhpStorm o un altro programma simile. Il terminale e la finestra dove scrivi comandi come `php --version`.

Installare PHP

Su macOS puoi usare Homebrew:

```
brew install php
```

Su Windows puoi installare PHP da php.net oppure usare un pacchetto locale come XAMPP. Su Linux di solito puoi usare il gestore pacchetti della distribuzione.

Non importa il metodo scelto: alla fine il comando `php` deve funzionare nel terminale.

Verificare l'installazione

Apri il terminale e scrivi:

```
php --version
```

Dovresti vedere una risposta simile:


```
PHP 8.3.0
```

Il numero può essere diverso. L'importante è che il terminale riconosca PHP e mostri una versione.

Scegliere una cartella di lavoro

Crea una cartella per gli esercizi, per esempio `corso-php`. Dentro questa cartella metterai i file `.php`.

```
mkdir corso-php
cd corso-php
```

`mkdir` crea la cartella. `cd` entra nella cartella.

Editor consigliato

Un buon editor ti aiuta con colori, indentazione e segnalazione degli errori. Visual Studio Code va benissimo per iniziare.

Suggerimento: salva sempre i file PHP con estensione `.php`, per esempio `index.php`. Senza questa estensione, PHP non sa che deve eseguire il file come codice.

Se il comando non funziona

Se il terminale risponde con un messaggio simile a `command not found` o `php non è riconosciuto`, PHP non è ancora raggiungibile dal terminale.

Questo non significa che hai sbagliato tutto. Di solito vuol dire una di queste cose:

- PHP non è stato installato
- il terminale va chiuso e riaperto
- il percorso di PHP non è stato aggiunto al `PATH`

Il `PATH` è una lista di cartelle in cui il terminale cerca i programmi. Quando scrivi `php`, il terminale guarda in quelle cartelle e prova a trovare l'eseguibile di PHP.

Piccola verifica finale

Crea un file chiamato `prova.php` con questo contenuto:

```
<?php  
echo "PHP funziona";
```

Poi esegilo dalla stessa cartella:

```
php prova.php
```

Se vedi `PHP funziona`, l'ambiente è pronto. Nel prossimo passo potrai concentrarti sul codice, non sull'installazione.

Operatori

Come usare operatori aritmetici, di confronto, logici e di assegnazione.

Cosa sono gli operatori

Gli operatori sono simboli che fanno qualcosa con i valori: sommano, confrontano, assegnano o combinano condizioni.

Operatori aritmetici

Servono per fare calcoli.

```
<?php  
$prezzo = 20;  
$sconto = 5;  
  
echo $prezzo - $sconto;
```

Operatori comuni:

- `+` somma
- `-` sottrazione
- `*` moltiplicazione
- `/` divisione
- `%` resto della divisione

```
<?php  
echo 10 % 3;
```

Output:

```
1
```

Assegnazione

`=` mette un valore in una variabile.

```
<?php  
$totale = 15;
```

Puoi aggiornare un valore in modo breve:

```
<?php  
$totale = 10;  
$totale += 5;  
  
echo $totale;
```

Confronto

Gli operatori di confronto producono `true` o `false` .

```
<?php
$eta = 18;
var_dump($eta >= 18);
```

Operatori comuni:

- `==` uguale come valore
- `===` uguale come valore e tipo
- `!=` diverso
- `>` maggiore
- `<` minore
- `>=` maggiore o uguale
- `<=` minore o uguale

Suggerimento: preferisci `===` quando vuoi un confronto preciso.

Operatori logici

Servono a unire condizioni.

```
<?php
$eta = 20;
$haBiglietto = true;

if ($eta >= 18 && $haBiglietto) {
    echo "Puoi entrare";
}
```

`&&` significa "e". `||` significa "oppure". `!` nega una condizione.

Le parentesi aiutano a rendere chiaro l'ordine dei controlli.

Precedenza: chi viene calcolato prima

PHP segue un ordine nei calcoli. Moltiplicazione e divisione vengono prima di somma e sottrazione.

```
<?php  
echo 2 + 3 * 4;
```

Output:

14

Prima viene calcolato `3 * 4`, poi viene aggiunto `2`.

Se vuoi un ordine diverso, usa le parentesi:

```
<?php  
echo (2 + 3) * 4;
```

Output:

20

Errore comune: usare = al posto di ===

`=` assegna un valore. `===` confronta due valori.

```
<?php
$eta = 18;

if ($eta === 18) {
    echo "Hai 18 anni";
}
```

Dentro una condizione quasi sempre vuoi confrontare, non assegnare.

Prova tu

Crea `$prezzo`, `$quantita` e `$sconto`. Calcola il totale usando almeno un operatore aritmetico e poi controlla con `if` se il totale è maggiore di 50.

Variabili e costanti

Come salvare valori in PHP con variabili e costanti.

Variabili

Una variabile è una specie di scatola con un'etichetta. Dentro puoi mettere un valore e riprenderlo più tardi.

In PHP le variabili iniziano con `$`.

```
<?php
$nome = "Luca";
$eta = 25;

echo $nome;
```

`$nome` contiene il testo `"Luca"`. `$eta` contiene il numero `25`.

Cambiare valore

Una variabile può cambiare durante il programma.

```
<?php
$totale = 10;
$totale = $totale + 5;

echo $totale;
```

Output:

15

PHP prende il vecchio valore di `$totale`, aggiunge `5` e salva il nuovo valore nella stessa variabile.

Nomi validi

Un nome di variabile può contenere lettere, numeri e `_`, ma non può iniziare con un numero.

```
<?php
$nomeUtente = "Sara";
$prezzo_finale = 19.90;
```

Scegli nomi chiari. `$prezzo` è meglio di `$p`.

Costanti

Una costante salva un valore che non dovrebbe cambiare.

```
<?php
const IVA = 22;

echo IVA;
```

Le costanti non usano `$` . Per convenzione spesso hanno nomi in maiuscolo.

Puoi anche usare `define` :

```
<?php
define("NOME_SITO", "Manuale PHP");

echo NOME_SITO;
```

Usa una variabile quando il valore può cambiare. Usa una costante quando il valore deve restare fisso.

Variabili nei calcoli

Le variabili diventano utili quando il valore viene riutilizzato più volte.

```
<?php
$prezzo = 40;
$ sconto = 8;
$totale = $prezzo - $sconto;

echo "Totale: $totale euro";
```

Se domani il prezzo cambia, modifichi solo `$prezzo` . Il calcolo continua a funzionare.

Errore comune: dimenticare il dollaro

In PHP `$nome` e `nome` non sono la stessa cosa.


```
<?php
$nome = "Luca";
echo nome;
```

Questo codice è sbagliato perché `nome` senza `$` non indica la variabile. Quando vuoi leggere o modificare una variabile, usa sempre il dollaro.

Scegliere tra `const` e `define`

Per iniziare, usa `const` quando dichiari costanti semplici nei tuoi file.

```
<?php
const COSTO_SPEDIZIONE = 4.90;
```

`define` esiste ancora e lo incontrerai in molti progetti, ma `const` è più leggibile nei casi base.

Prova tu

Crea tre variabili: `$nome`, `$prezzo` e `$quantita`. Calcola il totale e stampa una frase come `Luca deve pagare 30 euro`.

Array

Come raccogliere più valori in una sola variabile.

Cosa è un array

Un array è una variabile che contiene più valori. Pensalo come una fila di caselle numerate.

```
<?php
$frutti = ["mela", "pera", "banana"];
```

Ogni elemento ha una posizione, chiamata **indice**. In PHP il primo indice è `0` .

Leggere un elemento

```
<?php
$frutti = ["mela", "pera", "banana"];

echo $frutti[0];
```

Output:

```
mela
```

`$frutti[1]` contiene `"pera"` .

Aggiungere elementi

Puoi aggiungere un valore in fondo all'array con `[]` .

```
<?php
$frutti = ["mela", "pera"];
$frutti[] = "banana";

print_r($frutti);
```

`print_r` mostra una struttura in modo leggibile, utile mentre studi.

Contare elementi

```
<?php
$frutti = ["mela", "pera", "banana"];

echo count($frutti);
```

Output:

```
3
```

Scorrere un array

Con `foreach` puoi lavorare su ogni elemento.

```
<?php
$frutti = ["mela", "pera", "banana"];

foreach ($frutti as $frutto) {
    echo $frutto . "\n";
}
```

A ogni giro, `$frutto` contiene un valore diverso.

Modificare un elemento

Puoi cambiare un valore usando il suo indice.

```
<?php
$frutti = ["mela", "pera", "banana"];
$frutti[1] = "arancia";

print_r($frutti);
```

Ora al posto di `pera` trovi `arancia` .

Errore comune: dimenticare che si parte da zero

Se un array ha tre elementi, gli indici sono `0` , `1` e `2` .

```
<?php
$frutti = ["mela", "pera", "banana"];

echo $frutti[3];
```

`$frutti[3]` non esiste. Il terzo elemento è `$frutti[2]` .

A cosa servono davvero gli array

Gli array sono utili quando hai più valori dello stesso tipo: nomi, prezzi, voti, messaggi, prodotti.

```
<?php
$prezzi = [10, 20, 15];
$totale = 0;

foreach ($prezzi as $prezzo) {
    $totale += $prezzo;
}

echo $totale;
```

Qui l'array permette di calcolare il totale senza creare tre variabili separate.

Prova tu

Crea un array con tre nomi. Poi usa `foreach` per stampare `Ciao` seguito da ogni nome.

Array associativi

Come usare chiavi testuali per rappresentare dati strutturati.

Dalle posizioni ai nomi

Negli array indicizzati leggi i valori con numeri come `0` e `1`. Negli array associativi usi chiavi testuali.

```
<?php
$utente = [
    "nome" => "Luca",
    "eta" => 28,
    "email" => "luca@example.com",
];
```

Qui `"nome"`, `"eta"` ed `"email"` sono chiavi.

Leggere un valore

```
<?php
$utente = [
    "nome" => "Luca",
    "eta" => 28,
];

echo $utente["nome"];
```

Output:

Luca

Modificare un valore

```
<?php
$prodotto = [
    "nome" => "Quaderno",
    "prezzo" => 3.50,
];

$prodotto["prezzo"] = 3.20;
```

Stai cambiando solo il valore collegato alla chiave `"prezzo"` .

Aggiungere una chiave

```
<?php
$prodotto = [
    "nome" => "Quaderno",
];

$prodotto["disponibile"] = true;
```

Scorrere chiavi e valori

```
<?php
$utente = [
    "nome" => "Sara",
    "eta" => 31,
];

foreach ($utente as $chiave => $valore) {
    echo "$chiave: $valore\n";
}
```

Gli array associativi sono utili per rappresentare profili, prodotti, impostazioni e dati che hanno un nome preciso.

Un prodotto come array associativo

Quando i dati descrivono una stessa cosa, le chiavi rendono il codice piu leggibile.

```
<?php
$prodotto = [
    "nome" => "Penna",
    "prezzo" => 1.20,
    "disponibile" => true,
];

if ($prodotto["disponibile"]) {
    echo $prodotto["nome"] . " costa " . $prodotto["prezzo"] . " euro";
}
```

Leggendo le chiavi capisci subito che cosa rappresenta ogni valore.

Controllare se una chiave esiste

Prima di leggere una chiave che potrebbe mancare, puoi usare `isset` .

```
<?php
if (isset($utente["email"])) {
    echo $utente["email"];
}
```

Oppure puoi usare `??` per dare un valore predefinito.

```
<?php
$email = $utente["email"] ?? "email non disponibile";
```

Errore comune: usare il nome della chiave senza virgolette

```
<?php
echo $utente[nome];
```

Scrivi invece:

```
<?php
echo $utente["nome"];
```

Le virgolette fanno capire a PHP che `nome` è una chiave testuale.

Date e tempo

Come rappresentare date, orari e intervalli nei programmi PHP.

Perche le date richiedono attenzione

Una data sembra semplice, ma puo includere giorno, mese, anno, ora, minuti, secondi e fuso orario. PHP offre `DateTime` per gestirle in modo piu ordinato.

Creare una data

```
<?php
$soggi = new DateTime();

echo $soggi->format('d/m/Y');
```

`new DateTime()` crea un oggetto con data e ora attuali. `format` decide come mostrarlo.

Creare una data specifica

```
<?php
$scadenza = new DateTime('2026-06-15');

echo $scadenza->format('d/m/Y');
```

Output:

```
15/06/2026
```

Fusi orari

Il fuso orario dice a quale zona del mondo si riferisce l'orario.

```
<?php
$roma = new DateTime('now', new DateTimeZone('Europe/Rome'));

echo $roma->format('H:i');
```

Usare il fuso giusto evita risultati strani, soprattutto nei siti usati da persone in paesi diversi.

Timestamp

Un timestamp è un numero che rappresenta un momento nel tempo.

```
<?php
echo time();
```

È utile per confronti tecnici, ma per codice leggibile spesso `DateTime` è più chiaro.

Confrontare date

```
<?php
$oggi = new DateTime('2026-05-02');
$scadenza = new DateTime('2026-05-10');

if ($oggi < $scadenza) {
    echo "Sei ancora in tempo";
}
```

PHP confronta i due momenti e decide quale viene prima.

Aggiungere o togliere tempo

Con `modify` puoi spostare una data avanti o indietro.

```
<?php
$scadenza = new DateTime('2026-05-02');
$scadenza->modify('+7 days');

echo $scadenza->format('d/m/Y');
```

Output:

```
09/05/2026
```

Questo è utile per scadenze, promemoria e periodi di prova.

Differenza tra due date

```
<?php
$inizio = new DateTime('2026-05-02');
$fine = new DateTime('2026-05-10');

$differenza = $inizio->diff($fine);

echo $differenza->days;
```

Output:

```
8
```

`diff` restituisce un intervallo. La proprietà `days` contiene il numero totale di giorni tra le due date.

Errore comune: formato ambiguo

Date come `05/06/2026` possono essere lette come 5 giugno o 6 maggio, a seconda del contesto. Quando crei date nel codice, preferisci formati chiari come `2026-06-05`.

Stringhe

Come creare, combinare e manipolare testo in PHP.

Cosa è una stringa

Una stringa è testo: un nome, un messaggio, una email, una frase.

```
<?php
$nome = "Luca";
$saluto = 'Ciao';
```

Puoi usare virgolette doppie o singole.

Virgolette doppie e singole

Con le virgolette doppie, PHP legge le variabili dentro la stringa.

```
<?php
$nome = "Luca";
echo "Ciao $nome";
```

Output:

```
Ciao Luca
```

Con le virgolette singole, il testo resta letterale.

```
<?php
$nome = "Luca";
echo 'Ciao $nome';
```

Output:

```
Ciao $nome
```

Concatenazione

Per unire stringhe usa il punto `.`.

```
<?php
$nome = "Sara";
$messaggio = "Ciao " . $nome . "!";

echo $messaggio;
```

Funzioni comuni

PHP offre molte funzioni per lavorare con il testo.

```
<?php
$testo = "Manuale PHP";

echo strlen($testo);
```

`strlen` conta i caratteri in byte. Per testi con lettere accentate può servire `mb_strlen`, se l'estensione `mbstring` è disponibile.

Altre funzioni utili:

- `strtolower($testo)` porta in minuscolo
- `strtoupper($testo)` porta in maiuscolo
- `trim($testo)` toglie spazi all'inizio e alla fine
- `str_replace("PHP", "web", $testo)` sostituisce testo

Testo ricevuto dagli utenti

Quando mostri testo inserito da un utente in una pagina HTML, usa `htmlspecialchars` .

```
<?php
echo htmlspecialchars($nome, ENT_QUOTES, 'UTF-8');
```

Questo evita che testo pericoloso venga interpretato come HTML.

Leggere una parte di testo

Puoi prendere solo una parte di una stringa con `substr` .

```
<?php
$codice = "ORD-2026-15";
$prefisso = substr($codice, 0, 3);

echo $prefisso;
```

Output:

```
ORD
```

Il secondo numero indica da dove partire. Il terzo indica quanti caratteri prendere.

Controllare se un testo contiene una parola

```
<?php
$email = "luca@example.com";

if (str_contains($email, "@")) {
    echo "Sembra una email";
}
```

Questo non basta per validare davvero una email, ma mostra come controllare la presenza di una parte di testo.

Errore comune: dimenticare gli spazi nella concatenazione

```
<?php
$nome = "Luca";
echo "Ciao" . $nome;
```

Output:

CiaoLuca

Lo spazio va scritto esplicitamente:

```
<?php
echo "Ciao " . $nome;
```

Operazioni CRUD

Come creare, leggere, aggiornare e cancellare dati con PHP e SQL.

Cosa significa CRUD

CRUD indica le quattro operazioni principali sui dati:

- Create: creare
- Read: leggere
- Update: aggiornare
- Delete: cancellare

In un sito reale le usi continuamente: aggiungere un prodotto, mostrare una lista, modificare un profilo, eliminare un commento.

Create: inserire dati

```
<?php
$stmt = $pdo->prepare(
    "INSERT INTO prodotti (nome, prezzo) VALUES (:nome, :prezzo)"
);

$stmt->execute([
    "nome" => "Quaderno",
    "prezzo" => 3.50,
]);
```

Read: leggere dati

```
<?php
$stmt = $pdo->query("SELECT id, nome, prezzo FROM prodotti");
$prodotti = $stmt->fetchAll();

foreach ($prodotti as $prodotto) {
    echo $prodotto["nome"] . "\n";
}
```

`fetchAll` legge tutte le righe trovate.

Update: modificare dati

```
<?php
$stmt = $pdo->prepare(
    "UPDATE prodotti SET prezzo = :prezzo WHERE id = :id"
);

$stmt->execute([
    "prezzo" => 4.00,
    "id" => 1,
]);
```

WHERE è fondamentale: senza, rischi di modificare tutte le righe.

Delete: cancellare dati

```
<?php
$stmt = $pdo->prepare("DELETE FROM prodotti WHERE id = :id");
$stmt->execute(["id" => 1]);
```

Anche qui **WHERE** decide cosa cancellare.

Collegamento con le pagine web

Di solito un form raccoglie dati, PHP li valida, poi una query preparata li salva o li modifica.

Ordine consigliato:

1. ricevi i dati
2. controlla che siano validi
3. prepara la query
4. esegui la query
5. mostri un messaggio chiaro

Una piccola pagina di lettura

Una pagina che mostra prodotti potrebbe essere così:

```
<?php
$stmt = $pdo->query("SELECT id, nome, prezzo FROM prodotti ORDER BY nome");
$prodotti = $stmt->fetchAll();
?>

<ul>
    <?php foreach ($prodotti as $prodotto): ?>
        <li>
            <?= htmlspecialchars($prodotto["nome"], ENT_QUOTES, 'UTF-8') ?>
            - <?= $prodotto["prezzo"] ?> euro
        </li>
    <?php endforeach; ?>
</ul>
```

PHP legge i dati. HTML li mostra. `htmlspecialchars` protegge il testo prima di inserirlo nella pagina.

Confermare prima di cancellare

La cancellazione è l'operazione più delicata. In un'interfaccia reale conviene chiedere conferma e usare POST, non un semplice link GET.

```
<form method="post" action="elimina.php">
    <input type="hidden" name="id" value="1">
    <button type="submit">Elimina</button>
</form>
```

Riepilogo

CRUD non è una funzione speciale di PHP. È un modo per ricordare le azioni fondamentali sui dati. In PHP le realizzi con form, validazione, PDO e query preparate.

Collegarsi a un database con PDO

Come connettere PHP a un database usando PDO.

Cosa è PDO

PDO è uno strumento di PHP per parlare con database diversi usando un'interfaccia comune.

Puoi pensarlo come un adattatore: il tuo codice PHP usa PDO, e PDO si occupa di comunicare con il database.

Dati di connessione

Per collegarti servono:

- host, cioè dove si trova il database
- nome del database
- utente
- password
- charset, cioè il modo in cui vengono gestiti i caratteri

Primo collegamento

```
<?php
$dsn = "mysql:host=localhost;dbname=negozio;charset=utf8mb4";
$utente = "root";
$password = "";

$pdo = new PDO($dsn, $utente, $password);
```

`$dsn` descrive il tipo di database e dove trovarlo. In questo esempio usiamo MySQL.

Gestire gli errori

Aggiungi opzioni per ricevere errori chiari.

```
<?php
$pdo = new PDO($dsn, $utente, $password, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
```

`ERRMODE_EXCEPTION` fa lanciare eccezioni quando qualcosa va storto. `FETCH_ASSOC` restituisce righe come array associativi.

Proteggere le credenziali

Non mettere password reali in file pubblici o condivisi. Nei progetti veri si usano variabili d'ambiente o file di configurazione non caricati online.

Verificare il collegamento

```
<?php
echo "Connessione riuscita";
```

Se il codice arriva a questa riga senza errori, PHP è riuscito a collegarsi.

Gestire un errore di connessione

Una connessione può fallire: password sbagliata, database spento, nome del database errato. Per questo conviene usare `try` e `catch`.

```
<?php
try {
    $pdo = new PDO($dsn, $utente, $password, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);

    echo "Connessione riuscita";
} catch (PDOException $errore) {
    echo "Impossibile collegarsi al database";
}
```

Durante lo studio puoi leggere il messaggio tecnico dell'errore. In un sito pubblico, invece, mostra un messaggio generico e registra il dettaglio nei log.

Primo controllo con una query semplice

Dopo la connessione puoi provare una query piccola.

```
<?php
$stmt = $pdo->query("SELECT 1");
$risultato = $stmt->fetchColumn();

echo $risultato;
```

Se vedi **1**, PHP ha parlato con il database e ha ricevuto una risposta.

Errore comune: confondere PDO con SQL

PDO non sostituisce SQL. PDO è il mezzo con cui PHP invia istruzioni SQL al database e riceve risultati.

Query preparate

Come inviare query sicure al database evitando SQL injection.

Il problema

Quando una query contiene dati inseriti dall'utente, non devi incollarli direttamente nella stringa SQL.

Questo è pericoloso:

```
<?php
$sql = "SELECT * FROM utenti WHERE email = '$email'";
```

Se `$email` contiene testo malevolo, la query può cambiare significato. Questo rischio si chiama **SQL injection**.

Prepared statement

Una query preparata separa la struttura della query dai valori.

```
<?php
$sql = "SELECT * FROM utenti WHERE email = :email";
$stmt = $pdo->prepare($sql);
$stmt->execute(["email" => $email]);

$utente = $stmt->fetch();
```

`:email` è un placeholder, cioè un segnaposto. Il valore vero arriva dopo, in modo controllato.

SELECT sicura

```
<?php
$email = $_POST["email"] ?? "";

$stmt = $pdo->prepare("SELECT * FROM utenti WHERE email = :email");
$stmt->execute(["email" => $email]);

$utente = $stmt->fetch();

if ($utente) {
    echo "Utente trovato";
}
```

`fetch` legge una riga. Se non ci sono risultati, restituisce `false` .

INSERT sicuro

```
<?php
$stmt = $pdo->prepare(
    "INSERT INTO utenti (nome, email) VALUES (:nome, :email)"
);

$stmt->execute([
    "nome" => $nome,
    "email" => $email,
]);
```

Anche qui i valori non vengono incollati nella query.

Regola pratica

Ogni volta che una query usa dati esterni, usa `prepare` ed `execute` con placeholder. E una delle abitudini più importanti per scrivere PHP sicuro con i database.

Placeholder con punto interrogativo

Oltre ai placeholder con nome, puoi usare `?` .

```
<?php
$stmt = $pdo->prepare("SELECT * FROM utenti WHERE email = ?");
$stmt->execute([$email]);
```

Funziona bene per query piccole. I placeholder con nome sono spesso più leggibili quando i valori sono molti.

Leggere più righe

Per una lista di risultati usa `fetchAll` .

```
<?php
$stmt = $pdo->prepare("SELECT * FROM prodotti WHERE prezzo < :massimo");
$stmt->execute(["massimo" => 20]);

$prodotti = $stmt->fetchAll();

foreach ($prodotti as $prodotto) {
    echo $prodotto["nome"] . "\n";
}
```

La query resta sicura perché il valore `20` viene passato separatamente.

Errore comune: preparare e poi incollare lo stesso

Questo non va bene:

```
<?php
$stmt = $pdo->prepare("SELECT * FROM utenti WHERE email = '$email'");
```

Anche se usi `prepare` , hai comunque incollato `$email` nella stringa. Il valore deve entrare tramite placeholder.

Errori comuni

Come riconoscere gli errori PHP più frequenti e leggerne i messaggi.

Leggere un errore

Un messaggio di errore di solito dice tre cose:

1. che tipo di errore è
2. in quale file è successo
3. a quale riga guardare

La riga indicata non è sempre la causa esatta, ma è un ottimo punto da cui partire.

Parse error

Un parse error significa che PHP non riesce a capire la sintassi.

```
<?php  
echo "Ciao"
```

Qui manca il punto e virgola. PHP può mostrare un errore vicino alla riga successiva, perché si accorge tardi del problema.

Warning

Un warning segnala un problema, ma spesso il programma continua.

```
<?php  
include "file-che-non-esiste.php";
```

PHP avvisa che non trova il file.

Notice o warning su variabili mancanti

```
<?php
echo $nome;
```

Se `$nome` non è stata definita, PHP segnala il problema. Prima di usare una variabile, assicurati che abbia un valore.

```
<?php
$nome = $_GET["nome"] ?? "ospite";
```

TypeError

Un `TypeError` può comparire quando passi un tipo di dato sbagliato.

```
<?php
function somma(int $a, int $b): int {
    return $a + $b;
}

somma("ciao", 3);
```

La funzione si aspettava numeri interi, ma riceve una stringa.

Metodo pratico

Quando vedi un errore:

1. leggi il tipo di errore
2. apri il file indicato
3. guarda la riga indicata e quella prima
4. controlla parentesi, virgolette e punti e virgola
5. riduci il codice finché il problema diventa visibile

Esempio di lettura del messaggio

Un messaggio può assomigliare a questo:

```
Parse error: syntax error, unexpected token "echo" in index.php on line 5
```

Leggilo pezzo per pezzo:

- `Parse error` indica un problema di sintassi
- `unexpected token "echo"` dice che PHP ha trovato `echo` dove non se lo aspettava
- `index.php` è il file
- `line 5` è la riga da controllare

Spesso l'errore vero è nella riga precedente: un punto e virgola mancante, una parentesi non chiusa o una virgoletta dimenticata.

Warning non significa "ignorami"

Un warning può lasciare continuare il programma, ma va comunque capito. Se PHP avvisa che un file manca o che una variabile non esiste, il risultato della pagina potrebbe essere sbagliato.

Prova tu

Crea apposta un piccolo errore, per esempio togli un punto e virgola. Leggi il messaggio e prova a trovare file e riga. Poi correggi e riesegui il file.

Debugging

Come osservare cosa fa il codice PHP e trovare problemi passo dopo passo.

Cosa significa debugging

Fare debugging significa cercare e correggere un problema nel codice. Non è indovinare: è osservare con ordine.

var_dump

`var_dump` mostra valore e tipo.

```
<?php
$eta = "25";
var_dump($eta);
```

Output possibile:

```
string(2) "25"
```

Questo ti dice che `$eta` sembra un numero, ma in realta e una stringa.

print_r

`print_r` e comodo per array.

```
<?php
$utente = ["nome" => "Luca", "eta" => 28];
print_r($utente);
```

Output:

```
Array
(
    [nome] => Luca
    [eta] => 28
)
```

Mostrare gli errori durante lo studio

In ambiente di sviluppo puoi attivare la visualizzazione degli errori.

```
<?php
error_reporting(E_ALL);
ini_set('display_errors', '1');
```

Attenzione: su un sito pubblico non mostrare errori tecnici agli utenti. Possono rivelare dettagli interni del progetto.

Log degli errori

PHP può scrivere errori nei log del server. I log sono file dove vengono registrati problemi e informazioni utili.

Quando lavori su un hosting o un server, cerca la sezione dedicata agli error log.

Un metodo ordinato

Quando qualcosa non funziona:

1. riproduci il problema
2. controlla il messaggio di errore
3. stampa le variabili importanti con `var_dump`
4. verifica una riga alla volta cosa succede
5. toglie le stampe di debug quando hai finito

Il debugging migliora con la pratica. L'importante è fare un passo alla volta.

Restringere il punto del problema

Quando una pagina è lunga, non cercare tutto insieme. Metti piccole stampe in punti strategici.

```
<?php
echo "Prima del calcolo\n";

$totale = $prezzo * $quantita;

echo "Dopo il calcolo\n";
var_dump($totale);
```

Se vedi il primo messaggio ma non il secondo, il problema è tra i due punti.

Controllare i dati ricevuti

Molti problemi nascono da dati diversi da quelli che immagini.

```
<?php
var_dump($_POST);
```

Questa stampa ti mostra tutti i campi arrivati dal form. Usala solo durante lo sviluppo e rimuovila quando hai finito.

Diario del bug

Per problemi difficili, scrivi tre righe:

1. cosa mi aspettavo
2. cosa succede davvero
3. quale valore controllo adesso

Questo ti impedisce di cambiare codice a caso. Il debugging diventa una piccola indagine ordinata.

Eccezioni

Come gestire situazioni impreviste con try, catch e throw.

Cosa sono le eccezioni

Un'eccezione segnala che qualcosa è andato storto e il flusso normale non può continuare.

Non tutti gli errori sono uguali. Un file mancante può essere una situazione prevista da gestire. Una variabile scritta male è invece un bug da correggere.

try e catch

Metti nel blocco `try` il codice che potrebbe fallire. Nel blocco `catch` gestisci il problema.

```
<?php
try {
    $pdo = new PDO($dsn, $utente, $password);
    echo "Connessione riuscita";
} catch (PDOException $errore) {
    echo "Connessione non riuscita";
}
```

Se la connessione fallisce, PHP passa al `catch`.

throw

Puoi lanciare tu un'eccezione quando trovi una situazione non valida.

```
<?php
function dividi(float $a, float $b): float {
    if ($b === 0.0) {
        throw new InvalidArgumentException("Non puoi dividere per zero");
    }

    return $a / $b;
}
```

finally

`finally` viene eseguito comunque, sia se tutto va bene sia se c'è un'eccezione.

```
<?php
try {
    echo "Apro una risorsa";
} catch (Exception $errore) {
    echo "Errore";
} finally {
    echo "Pulizia finale";
}
```

Errore previsto o bug?

Usa le eccezioni per casi che il programma può incontrare e gestire: connessione fallita, file mancante, dato non valido.

Correggi invece i bug alla radice: nomi sbagliati, parentesi mancanti, logica errata. Un `catch` non deve nascondere codice rotto.

Recuperare il messaggio dell'eccezione

L'oggetto ricevuto nel `catch` contiene informazioni sull'errore.

```
<?php
try {
    throw new RuntimeException("File non trovato");
} catch (RuntimeException $errore) {
    echo $errore->getMessage();
}
```

Output:

```
File non trovato
```

Durante lo studio questo è utile. In produzione, invece, evita di mostrare dettagli tecnici agli utenti.

Eccezioni personalizzate

Per programmi piccoli puoi usare le eccezioni già pronte, come `RuntimeException` o `InvalidArgumentException`. Nei progetti più grandi puoi creare eccezioni con nomi specifici.

```
<?php
class ProdottoNonDisponibile extends RuntimeException {
}
```

Il nome rende più chiaro che tipo di problema stai gestendo.

Errore comune: catturare tutto e continuare

Un `catch` non deve far finta che vada tutto bene. Se non sai gestire davvero l'errore, registra il problema o mostra un messaggio chiaro. Nascondere l'errore rende il programma più difficile da correggere.

Break e continue

Come interrompere o saltare un passaggio dentro un ciclo.

Due comandi per controllare un ciclo

Dentro un ciclo puoi avere bisogno di fermarti prima o saltare un giro. PHP offre due parole utili:

- `break` interrompe il ciclo
- `continue` salta al giro successivo

Uscire con break

Immagina di cercare un nome in una lista. Quando lo trovi, non serve continuare.

```
<?php
$nomi = ["Luca", "Sara", "Marta"];

foreach ($nomi as $nome) {
    if ($nome === "Sara") {
        echo "Trovata Sara";
        break;
    }
}
```

Appena PHP incontra `break`, esce dal ciclo.

Saltare un giro con continue

`continue` non chiude il ciclo. Salta solo il resto del blocco corrente.

```
<?php
$numeri = [1, 2, 3, 4, 5];

foreach ($numeri as $numero) {
    if ($numero % 2 === 0) {
        continue;
    }

    echo $numero . "\n";
}
```

Output:

```
1
3
5
```

I numeri pari vengono saltati.

Quando usarli

`break` è utile quando hai già ottenuto quello che cercavi. `continue` è utile quando un elemento non ti interessa, ma vuoi controllare gli altri.

Attenzione: se usi troppi `break` e `continue`, il ciclo può diventare difficile da leggere. Usali quando rendono il codice più chiaro.

break in un ciclo for

`break` non funziona solo con `foreach`. Puoi usarlo anche con `for` e `while`.

```
<?php
for ($numero = 1; $numero <= 10; $numero++) {
    if ($numero === 4) {
        break;
    }

    echo $numero . "\n";
}
```

Output:

```
1
2
3
```

Quando `$numero` diventa `4`, il ciclo si ferma.

continue per filtrare dati

`continue` è utile quando vuoi ignorare alcuni elementi prima di fare il lavoro principale.

```
<?php
$nomi = ["Luca", "", "Sara"];

foreach ($nomi as $nome) {
    if ($nome === "") {
        continue;
    }

    echo "Ciao $nome\n";
}
```

La stringa vuota viene saltata. Il resto del ciclo lavora solo sui nomi validi.

Domanda guida

Prima di usare uno dei due comandi chiedi: voglio fermare tutta la ripetizione o saltare solo questo elemento? La risposta ti dice se scegliere `break` o `continue`.

Condizioni

Come far prendere decisioni a un programma PHP con `if`, `else` e `switch`.

A cosa servono le condizioni

Una condizione è un bivio. Se succede una cosa, il programma fa un'azione. Altrimenti ne fa un'altra.

if

`if` esegue un blocco solo quando la condizione è vera.

```
<?php
$eta = 20;

if ($eta >= 18) {
    echo "Sei maggiorenne";
}
```

La condizione sta tra parentesi tonde. Le istruzioni da eseguire stanno tra parentesi graffe.

else

`else` copre il caso contrario.

```
<?php
$eta = 16;

if ($eta >= 18) {
    echo "Sei maggiorenne";
} else {
    echo "Sei minorenni";
}
```

elseif

`elseif` permette di controllare più casi.

```
<?php
$voto = 7;

if ($voto >= 9) {
    echo "Ottimo";
} elseif ($voto >= 6) {
    echo "Sufficiente";
} else {
    echo "Da ripassare";
}
```

PHP controlla dall'alto verso il basso e si ferma al primo blocco valido.

switch

`switch` è comodo quando confronti lo stesso valore con più possibilità.

```
<?php
$giorno = "sabato";

switch ($giorno) {
    case "sabato":
    case "domenica":
        echo "Weekend";
        break;
    default:
        echo "Giorno lavorativo";
}
```

`break` evita che PHP continui nei casi successivi. `default` gestisce tutto quello che non è stato previsto prima.

Condizioni booleane

Una condizione deve diventare `true` o `false`.

```
<?php
$utenteAttivo = true;

if ($utenteAttivo) {
    echo "Accesso consentito";
}
```

Quando il nome è chiaro, il codice si legge quasi come una frase.

Combinare più controlli

Una decisione può dipendere da più condizioni.

```
<?php
$eta = 20;
$haDocumento = true;

if ($eta >= 18 && $haDocumento) {
    echo "Ingresso consentito";
} else {
    echo "Ingresso non consentito";
}
```

`&&` richiede che entrambe le condizioni siano vere. Se una delle due è falsa, PHP passa a `else` .

Rendere leggibile una condizione

Quando una condizione diventa lunga, puoi salvarne una parte in una variabile con un nome chiaro.

```
<?php
$totale = 75;
$clienteRegistrato = true;

$haDirittoSconto = $totale > 50 && $clienteRegistrato;

if ($haDirittoSconto) {
    echo "Sconto applicato";
}
```

Il programma fa la stessa cosa, ma è più facile da leggere.

Errore comune: troppi livelli annidati

Se metti un `if` dentro un `if` dentro un altro `if` , il codice diventa faticoso. Spesso puoi usare `elseif` , variabili booleane o funzioni piccole per chiarire la logica.

Prova tu

Scrivi un controllo per un ordine: se il totale è almeno 30 euro e il prodotto è disponibile, mostra `Ordine confermato` . Altrimenti mostra `Ordine non disponibile` .

Cicli

Come ripetere istruzioni con `while`, `do while`, `for` e `foreach`.

Cosa sono i cicli

Un ciclo ripete una o più istruzioni. È utile quando non vuoi scrivere la stessa riga molte volte.

while

`while` ripete finché una condizione resta vera.

```
<?php
$numero = 1;

while ($numero <= 3) {
    echo $numero . "\n";
    $numero++;
}
```

Output:

```
1
2
3
```

Senza `$numero++`, la condizione resterebbe sempre vera e il ciclo non finirebbe.

do while

`do while` esegue il blocco almeno una volta, poi controlla la condizione.


```
<?php
$tentativi = 0;

do {
    echo "Provo...\n";
    $tentativi++;
} while ($tentativi < 1);
```

for

for è comodo quando sai quante ripetizioni vuoi fare.

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Riga $i\n";
}
```

Dentro le parentesi ci sono: valore iniziale, condizione, aggiornamento.

foreach

foreach serve per scorrere array.

```
<?php
$nomi = ["Luca", "Sara", "Marta"];

foreach ($nomi as $nome) {
    echo "Ciao $nome\n";
}
```

A ogni giro, **\$nome** contiene un elemento diverso dell'array.

Scegliere il ciclo giusto

Usa `foreach` per gli array. Usa `for` quando conosci il numero di ripetizioni. Usa `while` quando vuoi ripetere finché una condizione cambia.

Evitare cicli infiniti

Un ciclo infinito continua per sempre. Di solito succede quando la condizione non cambia mai.

```
<?php
$numero = 1;

while ($numero <= 3) {
    echo $numero;
}
```

Qui `$numero` resta sempre `1`, quindi la condizione resta vera. Per correggerlo devi aggiornare il valore dentro il ciclo.

```
<?php
$numero = 1;

while ($numero <= 3) {
    echo $numero . "\n";
    $numero++;
}
```

Costruire un totale con foreach

Un uso molto comune dei cicli è sommare valori.

```
<?php
$prezzi = [10, 15, 5];
$totale = 0;

foreach ($prezzi as $prezzo) {
    $totale += $prezzo;
}

echo $totale;
```

Output:

30

Prova tu

Crea un array di voti e usa `foreach` per sommarli. Poi dividi il totale per `count($voti)` e stampa la media.

Funzioni

Come organizzare il codice PHP in blocchi riutilizzabili.

Cosa è una funzione

Una funzione è un blocco di codice con un nome. La scrivi una volta e puoi usarla più volte.

E come una ricetta: invece di riscrivere tutti i passaggi, richiami la ricetta quando serve.

Dichiarare una funzione

```
<?php
function saluta() {
    echo "Ciao!";
}
```

Questo codice crea la funzione, ma non la esegue ancora.

Chiamare una funzione

Per eseguirla, scrivi il suo nome seguito da parentesi.

```
<?php
function saluta() {
    echo "Ciao!";
}

saluta();
```

Output:

```
Ciao!
```

Riutilizzare codice

Una funzione evita ripetizioni.

```
<?php
function mostraSeparatore() {
    echo "---\n";
}

mostraSeparatore();
echo "Prodotti\n";
mostraSeparatore();
echo "Totale\n";
mostraSeparatore();
```

Se un giorno vuoi cambiare il separatore, modifichi una sola funzione.

Nomi chiari

Il nome deve dire cosa fa la funzione.

```
<?php
function calcolaTotale() {
    echo 10 + 5;
}
```

`calcolaTotale` è più utile di `faiCosa` .

Suggerimento: una funzione dovrebbe fare una cosa principale. Se fa troppe cose, diventa difficile da capire e da riusare.

Una funzione con un compito chiaro

Immagina di dover mostrare più volte il prezzo con la parola `euro` .

```
<?php
function mostraPrezzo($prezzo) {
    echo $prezzo . " euro";
}

mostraPrezzo(12);
```

La funzione non conosce tutti i prodotti del programma. Si occupa solo di mostrare un prezzo. Questo la rende facile da capire.

Prima senza funzione, poi con funzione

All'inizio potresti scrivere:

```
<?php
echo "Ciao Luca\n";
echo "Ciao Sara\n";
```

Con una funzione elimini la ripetizione:

```
<?php
function saluta($nome) {
    echo "Ciao $nome\n";
}

saluta("Luca");
saluta("Sara");
```

Il vantaggio non è scrivere meno righe a tutti i costi. Il vantaggio è dare un nome a un'azione.

Errore comune: chiamare la funzione con un nome diverso

```
<?php
function saluta() {
    echo "Ciao";
}

saluto();
```

`saluta` e `saluto` sono due nomi diversi. PHP cercherà una funzione che non esiste.

Parametri e valori di ritorno

Come passare dati a una funzione e ricevere un risultato.

Parametri e argomenti

Una funzione può ricevere dati dall'esterno. I nomi scritti nella definizione si chiamano **parametri**.

```
<?php
function saluta($nome) {
    echo "Ciao $nome";
}

saluta("Luca");
```

`$nome` è il parametro. `"Luca"` è l'argomento passato quando chiami la funzione.

Piu parametri

```
<?php
function somma($a, $b) {
    echo $a + $b;
}

somma(3, 4);
```

Output:

7

return

return restituisce un valore a chi ha chiamato la funzione.

```
<?php
function somma($a, $b) {
    return $a + $b;
}

$totale = somma(3, 4);
echo $totale;
```

La funzione non stampa direttamente. Calcola e consegna il risultato.

Valori predefiniti

Puoi dare un valore di default a un parametro.


```
<?php
function saluta($nome = "ospite") {
    return "Ciao $nome";
}

echo saluta();
```

Output:

```
Ciao ospite
```

Type declarations di base

Puoi indicare che tipo di dato ti aspetti.

```
<?php
function calcolaIva(float $prezzo): float {
    return $prezzo * 0.22;
}
```

`float $prezzo` dice che il parametro deve essere un numero decimale. `: float` dice che la funzione restituisce un numero decimale.

Queste indicazioni rendono gli errori più facili da trovare.

echo o return?

Usa `echo` quando vuoi mostrare qualcosa subito. Usa `return` quando vuoi consegnare un valore al resto del programma.

```
<?php
function creaSaluto($nome) {
    return "Ciao $nome";
}

$messaggio = creaSaluto("Luca");
echo $messaggio;
```

Questa funzione è più flessibile: puoi stampare il messaggio, salvarlo, mandarlo in una email o combinarlo con altro testo.

Parametri con nomi chiari

I parametri sono variabili locali della funzione. Scegli nomi che spiegano il loro ruolo.

```
<?php
function calcolaTotale($prezzo, $quantita) {
    return $prezzo * $quantita;
}
```

`$prezzo` e `$quantita` sono più chiari di `$a` e `$b`, tranne negli esempi matematici molto piccoli.

Errore comune: dimenticare di usare il valore di ritorno

```
<?php
function somma($a, $b) {
    return $a + $b;
}

somma(2, 3);
```

La funzione calcola `5`, ma il valore viene perso. Se ti serve, salvalo o stampalo:

```
<?php
$totale = somma(2, 3);
echo $totale;
```

Scope delle variabili

Dove esistono le variabili e come evitare confusione tra funzioni e codice esterno.

Cosa significa scope

Lo **scope** è la zona in cui una variabile esiste ed è leggibile.

Una variabile creata dentro una funzione non è automaticamente disponibile fuori. Una variabile creata fuori non entra automaticamente dentro.

Variabili locali

```
<?php
function saluta() {
    $nome = "Luca";
    echo $nome;
}

saluta();
```

`$nome` esiste solo dentro la funzione `saluta` .

Questo codice invece non funziona come potresti aspettarti:

```
<?php
function preparaNome() {
    $nome = "Luca";
}

preparaNome();
echo $nome;
```

Fuori dalla funzione, `$nome` non esiste.

Parametri

Il modo più pulito per portare dati dentro una funzione e usare parametri.

```
<?php
function saluta($nome) {
    echo "Ciao $nome";
}

saluta("Sara");
```

Valori di ritorno

Il modo più pulito per portare dati fuori da una funzione e usare `return`.

```
<?php
function creaMessaggio($nome) {
    return "Ciao $nome";
}

$messaggio = creaMessaggio("Marta");
echo $messaggio;
```

Variabili globali

PHP permette di usare `global`, ma è meglio limitarlo.

```
<?php
$iva = 22;

function mostraIva() {
    global $iva;
    echo $iva;
}
```

Il codice funziona, ma la funzione dipende da qualcosa che sta fuori. Con parametri e `return`, il collegamento è più chiaro.

Perché lo scope aiuta

Lo scope evita che una variabile venga modificata per sbaglio da una parte lontana del programma. Una funzione con parametri chiari è più prevedibile.

```
<?php
function applicaSconto($prezzo, $sconto) {
    return $prezzo - $sconto;
}

$totale = applicaSconto(30, 5);
echo $totale;
```

La funzione lavora solo con i valori che riceve. Non deve cercare variabili nascoste nel resto del file.

Stesso nome, scope diverso

Puoi avere variabili con lo stesso nome in scope diversi.

```
<?php
$nome = "Luca";

function saluta() {
    $nome = "Sara";
    echo $nome;
}

saluta();
echo $nome;
```

Dentro la funzione `$nome` vale `Sara` . Fuori vale ancora `Luca` .

Regola pratica

Quando scrivi una funzione, prova a farle ricevere tutto cio che le serve tramite parametri. Se deve produrre un risultato, usa `return` . Tieni `global` come ultima scelta.

Classi e oggetti

I concetti essenziali della programmazione a oggetti in PHP.

Perche usare gli oggetti

Quando un programma cresce, variabili e funzioni sparse possono diventare difficili da seguire. La programmazione a oggetti aiuta a tenere insieme dati e comportamenti collegati.

Classe e oggetto

Una **classe** e un modello. Un **oggetto** e una copia concreta creata da quel modello.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;
}

$quaderno = new Prodotto();
$quaderno->nome = "Quaderno";
$quaderno->prezzo = 3.50;

echo $quaderno->nome;
```

`new Prodotto()` crea un oggetto.

Proprieta

Le proprieta sono variabili dentro una classe.

```
public string $nome;
public float $prezzo;
```

`public` significa che puoi leggere e modificare la proprieta dall'esterno.

Metodi

I metodi sono funzioni dentro una classe.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;

    public function descrizione(): string {
        return $this->nome . " costa " . $this->prezzo . " euro";
    }
}

$prodotto = new Prodotto();
$prodotto->nome = "Penna";
$prodotto->prezzo = 1.20;

echo $prodotto->descrizione();
```

`$this` indica l'oggetto corrente.

Quando iniziare a usarli

All'inizio puoi scrivere programmi semplici con variabili, array e funzioni. Gli oggetti diventano utili quando vuoi rappresentare concetti del progetto: utenti, prodotti, ordini, articoli.

Oggetti e array associativi

Un array associativo può descrivere un prodotto:

```
<?php
$prodotto = [
    "nome" => "Penna",
    "prezzo" => 1.20,
];
```

Una classe rende esplicito che quei dati appartengono a un concetto preciso.


```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;
}
```

La classe dice: un prodotto ha un nome e un prezzo. Questo aiuta quando il progetto cresce.

Creare piu oggetti

```
<?php
$penna = new Prodotto();
$penna->nome = "Penna";
$penna->prezzo = 1.20;

$quaderno = new Prodotto();
$quaderno->nome = "Quaderno";
$quaderno->prezzo = 3.50;
```

`$penna` e `$quaderno` sono due oggetti diversi creati dallo stesso modello.

Errore comune: dimenticare new

Per creare un oggetto devi usare `new` .

```
<?php
$prodotto = new Prodotto();
```

Senza `new` , non stai creando un'istanza della classe.

Costruttori e visibilit 

Come inizializzare oggetti e controllare l'accesso a propriet  e metodi.

Il costruttore

Il costruttore   un metodo speciale chiamato automaticamente quando crei un oggetto.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;

    public function __construct(string $nome, float $prezzo) {
        $this->nome = $nome;
        $this->prezzo = $prezzo;
    }
}

$prodotto = new Prodotto("Quaderno", 3.50);
```

`__construct` prepara l'oggetto con i dati iniziali.

Visibilit 

La visibilit  decide da dove puoi usare propriet  e metodi.

- `public` : accessibile anche dall'esterno
- `private` : accessibile solo dentro la classe
- `protected` : accessibile dentro la classe e nelle classi figlie

Private

```
<?php
class Conto {
    private float $saldo = 0;

    public function deposita(float $importo): void {
        if ($importo > 0) {
            $this->saldo += $importo;
        }
    }

    public function leggiSaldo(): float {
        return $this->saldo;
    }
}
```

`$saldo` è privato: dall'esterno non puoi modificarlo direttamente. Devi passare dai metodi.

Incapsulamento

L'incapsulamento significa proteggere i dati interni di un oggetto e offrire metodi chiari per usarli.

Questo aiuta a evitare valori impossibili, come un deposito negativo.

Getter e setter

Un **getter** legge un valore. Un **setter** lo modifica controllando prima se va bene.

```
public function leggiSaldo(): float {
    return $this->saldo;
}
```

Non servono sempre, ma sono utili quando vuoi controllare l'accesso ai dati.

Promozione delle proprietà nel costruttore

In PHP moderno puoi scrivere costruttori più compatti.

```
<?php
class Prodotto {
    public function __construct(
        public string $nome,
        private float $prezzo
    ) {
    }
}
```

Questa sintassi crea le proprietà e le valorizza nello stesso punto. È comoda, ma all'inizio va letta con calma:

`public string $nome` dentro il costruttore diventa una proprietà pubblica della classe.

Setter con controllo

Un setter serve quando vuoi permettere una modifica, ma solo se il valore è valido.

```
<?php
class Prodotto {
    private float $prezzo;

    public function cambiaPrezzo(float $prezzo): void {
        if ($prezzo <= 0) {
            return;
        }

        $this->prezzo = $prezzo;
    }
}
```

Qui impedisce prezzi uguali o minori di zero.

Regola pratica

Usa `private` per i dati che non devono essere modificati liberamente. Offri metodi pubblici quando vuoi controllare come quei dati vengono letti o cambiati.

Ereditarieta, interfacce e trait

Come riusare e organizzare comportamento tra classi PHP.

Prima la soluzione semplice

Ereditarieta, interfacce e trait sono strumenti utili, ma non servono in ogni programma. Prima prova con classi piccole e metodi chiari. Usa questi strumenti quando risolvono un problema reale.

Ereditarieta con extends

L'ereditarieta permette a una classe di partire da un'altra classe.

```
<?php
class Utente {
    public function saluta(): string {
        return "Ciao";
    }
}

class Admin extends Utente {
    public function pannello(): string {
        return "Area amministrazione";
    }
}

$admin = new Admin();
echo $admin->saluta();
```

`Admin` eredita il metodo `saluta` da `Utente` .

Interfacce con implements

Un'interfaccia descrive quali metodi una classe deve avere.

```
<?php
interface Notificabile {
    public function invia(string $messaggio): void;
}

class Email implements Notificabile {
    public function invia(string $messaggio): void {
        echo "Email: $messaggio";
    }
}
```

L'interfaccia non spiega come fare il lavoro. Stabilisce il contratto.

Trait

Un trait permette di riusare metodi in più classi.

```
<?php
trait LoggaAzioni {
    public function log(string $messaggio): void {
        echo "Log: $messaggio";
    }
}

class Ordine {
    use LoggaAzioni;
}
```

Quando usarli

Usa `extends` quando una classe è davvero una versione più specifica di un'altra. Usa un'interfaccia quando classi diverse devono rispettare lo stesso contratto. Usa un trait per condividere piccoli comportamenti, senza costruire una gerarchia complicata.

Un esempio di interfaccia con piu classi

Il valore di un'interfaccia si vede quando classi diverse possono essere usate nello stesso modo.

```
<?php
interface Pagabile {
    public function paga(float $importo): void;
}

class CartaCredito implements Pagabile {
    public function paga(float $importo): void {
        echo "Pagamento con carta: $importo";
    }
}

class Bonifico implements Pagabile {
    public function paga(float $importo): void {
        echo "Pagamento con bonifico: $importo";
    }
}
```

Entrambe le classi promettono di avere un metodo `paga` .

Quando evitare l'ereditarieta

Non usare `extends` solo per riusare due righe di codice. L'ereditarieta crea un legame forte tra classi. Se il legame non rappresenta davvero un rapporto "e un tipo di", una funzione, una composizione o un trait piccolo possono essere piu semplici.

Riepilogo

Questi strumenti servono a organizzare progetti piu grandi. Se una pagina con classi semplici e metodi chiari risolve il problema, resta su quella strada. La semplicita e una scelta tecnica, non una mancanza.

Autoload e namespace

Come organizzare file, classi e nomi in progetti PHP piu grandi.

Il problema dei file

In un progetto piccolo puoi usare `require` per includere file a mano. In un progetto più grande diventa scomodo.

L'autoload carica le classi quando servono, senza scrivere `require` per ogni file.

Namespace

Un namespace è uno spazio dei nomi. Serve a evitare conflitti tra classi con lo stesso nome.

```
<?php
namespace App\Model;

class Prodotto {
}
```

Questa classe si chiama davvero `App\Model\Prodotto`.

use

Per usare una classe con namespace puoi importarla con `use`.

```
<?php
use App\Model\Prodotto;

$prodotto = new Prodotto();
```

Senza `use`, dovresti scrivere il nome completo.

Struttura delle cartelle

Una struttura comune è:


```
progetto/  
  src/  
    Model/  
      Prodotto.php  
  vendor/  
    composer.json
```

Il file `Prodotto.php` contiene la classe `App\Model\Prodotto` .

Autoload con Composer

Nel `composer.json` puoi configurare l'autoload:

```
{  
  "autoload": {  
    "psr-4": {  
      "App\\": "src/"  
    }  
  }  
}
```

Poi esegui:

```
composer dump-autoload
```

Nel codice principale carichi l'autoload:

```
<?php  
require __DIR__ . '/vendor/autoload.php';
```

Nomi coerenti

Usa nomi di classi chiari e una cartella coerente con il namespace. Questo rende il progetto più facile da navigare.

Un file, una classe principale

Una convenzione semplice e mettere una classe principale per file.

```
<?php
namespace App\Model;

class Prodotto {
}
```

Questo file potrebbe chiamarsi `src/Model/Prodotto.php`. Il percorso rispecchia il namespace.

Perche serve dump-autoload

Quando cambi la configurazione dell'autoload in `composer.json`, Composer deve rigenerare le informazioni interne.

```
composer dump-autoload
```

Se aggiungi una nuova classe seguendo la struttura già configurata, spesso non serve fare altro. Se invece cambi la mappa dei namespace, esegui questo comando.

Errore comune: namespace e cartella incoerenti

Se dichiari:

```
<?php
namespace App\Model;
```

ma metti il file in una cartella non collegata ad `App\`, Composer potrebbe non trovarlo. Quando una classe non viene caricata, controlla nome del namespace, nome della classe, percorso del file e configurazione `psr-4`.

Composer

Come installare librerie PHP e gestire dipendenze di progetto.

Cosa è Composer

Composer è il gestore di dipendenze più usato in PHP. Una dipendenza è una libreria esterna che il tuo progetto usa per non riscrivere tutto da zero.

Esempio: invece di scrivere da solo una libreria per inviare email, puoi installarne una già pronta.

composer.json

Il file `composer.json` descrive il progetto e le librerie richieste.

Puoi crearne uno con:

```
composer init
```

Per iniziare puoi anche installare direttamente un pacchetto.

```
composer require monolog/monolog
```

Composer aggiorna `composer.json` e scarica il codice nella cartella `vendor`.

Installare dipendenze

Se scarichi un progetto che ha già `composer.json`, esegui:

```
composer install
```

Composer legge anche `composer.lock`, se presente, e installa le versioni previste.

Autoload

Composer crea un file di autoload che carica classi e librerie.

```
<?php  
require __DIR__ . '/vendor/autoload.php';
```

Dopo questa riga puoi usare le librerie installate.

Aggiornare con cautela

Per aggiornare le librerie:

```
composer update
```

Questo comando può cambiare versioni. Nei progetti reali va usato con attenzione e seguito da test.

Regola pratica

- `composer require` aggiunge una libreria
- `composer install` installa ciò che il progetto richiede
- `composer update` aggiorna le versioni consentite

Composer diventa importante quando il progetto smette di essere un singolo file e inizia a usare librerie esterne.

composer.lock

Quando installi dipendenze, Composer crea spesso anche `composer.lock`. Questo file registra le versioni esatte installate.

Il suo scopo è semplice: se un'altra persona installa il progetto, riceve le stesse versioni delle librerie. Così si evitano differenze difficili da capire.

Cartella vendor

La cartella `vendor` contiene il codice scaricato da Composer. Di solito non la modifichi a mano.

In molti progetti `vendor` non viene salvata in Git. Si salva invece `composer.json` e `composer.lock`, poi ogni persona esegue:

```
composer install
```

Usare una libreria installata

Dopo l'autoload, puoi usare classi fornite dai pacchetti.

```
<?php
require __DIR__ . '/vendor/autoload.php';

// Qui puoi usare le librerie installate con Composer
```

Ogni libreria ha la propria documentazione, ma il punto di ingresso nel progetto PHP resta quasi sempre `vendor/autoload.php`.

Regola pratica

Non installare librerie per ogni piccola cosa. Prima capisci il problema. Poi scegli una libreria quando ti evita lavoro complesso o delicato.

Test e buone pratiche

Come scrivere PHP più affidabile, leggibile e facile da mantenere.

Scrivere codice leggibile

Il primo passo per avere codice affidabile e renderlo leggibile.

Scegli nomi chiari:

```
<?php
$prezzoFinale = 18.50;
```

E meglio di:

```
<?php
$pf = 18.50;
```

Funzioni piccole

Una funzione piccola e piu facile da capire e provare.

```
<?php
function calcolaSconto(float $prezzo): float {
    if ($prezzo > 50) {
        return 10;
    }

    return 0;
}
```

Questa funzione fa una cosa: decide lo sconto.

Validare gli input

I dati che arrivano da form, URL o file esterni vanno controllati.

```
<?php
$email = $_POST["email"] ?? "";

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email non valida";
    exit;
}
```

La validazione evita errori e migliora la sicurezza.

Un test automatico semplice

Un test automatico verifica che una parte del codice funzioni come previsto.

```
<?php
function somma(int $a, int $b): int {
    return $a + $b;
}

if (somma(2, 3) !== 5) {
    echo "Test fallito";
    exit(1);
}

echo "Test riuscito";
```

Nei progetti veri si usano strumenti come PHPUnit, ma l'idea è la stessa: controllare il risultato atteso.

Sicurezza essenziale

Abitudini importanti:

- valida i dati in ingresso
- usa `htmlspecialchars` quando stampi dati in HTML
- usa query preparate per il database
- non salvare password in chiaro
- non mostrare errori tecnici in produzione

La sicurezza non è un capitolo separato da ricordare alla fine. È una serie di piccole scelte in ogni pagina.

Rendere un test piu leggibile

Anche senza strumenti esterni puoi separare preparazione, azione e controllo.

```
<?php
function calcolaTotale(float $prezzo, int $quantita): float {
    return $prezzo * $quantita;
}

$risultato = calcolaTotale(5, 3);

if ($risultato !== 15.0) {
    echo "Test fallito: totale non corretto";
    exit(1);
}

echo "Test riuscito";
```

Il test dice chiaramente cosa si aspettava.

Buone pratiche quotidiane

Non servono regole complicate per migliorare subito il codice. Parti da queste:

- una variabile deve avere un nome comprensibile
- una funzione deve avere un compito principale
- i dati esterni vanno controllati prima di usarli
- gli errori vanno letti, non ignorati
- il codice duplicato spesso merita una funzione

Prima di considerare finita una pagina

Prova almeno un caso normale, un caso vuoto e un caso sbagliato. Per esempio, se ricevi un'email, prova un'email valida, un campo vuoto e un testo che non e un'email.

Upload di file

Come ricevere file da un form e gestirli in modo controllato.

Preparare il form

Per inviare file, il form deve usare `method="post"` e `enctype="multipart/form-data"` .

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  <input type="file" name="documento">
  <button type="submit">Carica</button>
</form>
```

Senza `enctype` , il file non arriva correttamente a PHP.

Leggere il file con `$_FILES`

In `upload.php` :

```
<?php
$file = $_FILES["documento"] ?? null;

if ($file === null) {
    echo "Nessun file inviato";
    exit;
}

print_r($file);
```

`$_FILES` contiene informazioni come nome originale, tipo, dimensione, percorso temporaneo ed eventuale errore.

Controllare gli errori

```
<?php
if ($_FILES["documento"]["error"] !== UPLOAD_ERR_OK) {
    echo "Errore durante il caricamento";
    exit;
}
```

Controlla sempre l'errore prima di usare il file.

Controllare dimensione e tipo

```
<?php
$dimensioneMassima = 2 * 1024 * 1024;

if ($_FILES["documento"]["size"] > $dimensioneMassima) {
    echo "File troppo grande";
    exit;
}
```

Per il tipo, non fidarti solo del nome del file. Usa controlli server-side e accetta solo formati previsti.

Salvare il file

```
<?php
$cartella = __DIR__ . "/uploads";
$destinazione = $cartella . "/documento.pdf";

move_uploaded_file($_FILES["documento"]["tmp_name"], $destinazione);
```

`move_uploaded_file` sposta il file dalla cartella temporanea alla cartella scelta.

Attenzione: non salvare file caricati dagli utenti con nomi non controllati e non permettere l'esecuzione di file caricati. L'upload è una zona delicata della sicurezza web.

Creare la cartella di destinazione

Prima di spostare un file, assicurati che la cartella esista.

```
<?php
$cartella = __DIR__ . "/uploads";

if (!is_dir($cartella)) {
    mkdir($cartella);
}
```

`__DIR__` indica la cartella del file PHP corrente. In questo modo costruisci un percorso stabile.

Evitare nomi pericolosi

Il nome originale del file arriva dal computer dell'utente, quindi non va usato alla cieca.

```
<?php
$nomeSicuro = uniqid("file_", true) . ".pdf";
$destinazione = $cartella . "/" . $nomeSicuro;
```

`uniqid` crea un nome abbastanza unico per evitare sovrascritture accidentali. In un progetto reale controlleresti anche il tipo effettivo del file.

Checklist minima

Prima di salvare un file caricato, controlla sempre:

- che non ci siano errori di upload
- che la dimensione sia accettabile
- che il tipo sia tra quelli consentiti
- che il nome finale sia deciso dal server

- che la cartella di upload non esegua codice PHP

Form, GET e POST

Come ricevere dati inviati da un form HTML in PHP.

Cosa fa un form

Un form HTML permette a una persona di inserire dati e inviarli al server. PHP può leggere quei dati e usarli.

Un form con GET

```
<form action="cerca.php" method="get">
  <input type="text" name="q">
  <button type="submit">Cerca</button>
</form>
```

`action` indica quale file riceve i dati. `method` indica come vengono inviati.

In `cerca.php` puoi leggere il campo così:

```
<?php
$ricerca = $_GET["q"] ?? "";
echo "Hai cercato: " . htmlspecialchars($ricerca, ENT_QUOTES, 'UTF-8');
```

`$_GET` contiene i dati inviati con metodo GET. `?? ""` evita errori se il campo non esiste.

Quando usare GET

GET mette i dati nell'indirizzo della pagina. Va bene per ricerche e filtri.

Esempio:

```
cerca.php?q=php
```

Non usarlo per password o dati sensibili.

Un form con POST

```
<form action="contatto.php" method="post">
  <input type="text" name="nome">
  <textarea name="messaggio"></textarea>
  <button type="submit">Invia</button>
</form>
```

In `contatto.php` :

```
<?php
$nome = $_POST["nome"] ?? "";
$messaggio = $_POST["messaggio"] ?? "";

echo "Grazie, " . htmlspecialchars($nome, ENT_QUOTES, 'UTF-8');
```

`$_POST` contiene i dati inviati con metodo POST.

Validare sempre

Non fidarti mai dei dati ricevuti da un form. Controlla che ci siano, che abbiano il formato giusto e che non siano troppo lunghi.

```
<?php
if ($nome === "") {
    echo "Il nome e obbligatorio";
}
```

La validazione rende il programma piu sicuro e piu gentile con chi lo usa.

Mantenere il valore dopo l'invio

Quando un form contiene errori, è gentile mostrare di nuovo i valori inseriti.

```
<?php
$nome = $_POST["nome"] ?? "";

?>

<form method="post">
    <input type="text" name="nome" value="<?= htmlspecialchars($nome,
ENT_QUOTES, 'UTF-8') ?>">
    <button type="submit">Invia</button>
</form>
```

Se il form viene inviato, `$nome` contiene il valore ricevuto. `htmlspecialchars` lo rende sicuro prima di rimetterlo nell'HTML.

Controllare il metodo della richiesta

Una pagina può essere aperta normalmente o ricevere un invio dal form. Puoi distinguere i due casi con `$_SERVER["REQUEST_METHOD"]`.

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nome = $_POST["nome"] ?? "";

    if ($nome === "") {
        echo "Il nome è obbligatorio";
    }
}
```

In questo modo validi i dati solo quando il form è stato inviato.

Regola pratica

Usa GET quando il risultato può stare nell'indirizzo, come una ricerca. Usa POST quando stai inviando dati che cambiano qualcosa o che non vuoi mostrare nell'URL.

PHP e HTML

Come mescolare PHP e HTML per generare pagine dinamiche.

Due ruoli diversi

HTML descrive la pagina. PHP prepara dati e pezzi dinamici della pagina.

```
<?php
$titolo = "Benvenuto";
?>

<h1><?php echo $titolo; ?></h1>
```

PHP calcola il valore. HTML lo mostra.

Echo dentro HTML

Per stampare un valore puoi usare `echo` .

```
<?php
$nome = "Luca";
?>

<p>Ciao, <?php echo $nome; ?>!</p>
```

Esiste anche una forma breve:

```
<p>Ciao, <?= $nome ?>!</p>
```

`<?= $nome ?>` significa: stampa questo valore.

Separare logica e markup

Cerca di preparare i dati prima dell'HTML.

```
<?php
$utente = [
    "nome" => "Sara",
    "ruolo" => "amministratrice",
];
?>

<h1><?= $utente["nome"] ?></h1>
<p>Ruolo: <?= $utente["ruolo"] ?></p>
```

Questo rende la pagina più leggibile: prima organizziamo i dati, poi li mostriamo.

Scorrere dati in una pagina

```
<?php
$prodotti = ["Penna", "Quaderno", "Zaino"];
?>

<ul>
    <?php foreach ($prodotti as $prodotto): ?>
        <li><?= $prodotto ?></li>
    <?php endforeach; ?>
</ul>
```

La sintassi con `endforeach` è comoda nei template HTML per evitare troppe parentesi graffe.

Proteggere il testo

Quando stampi dati che arrivano da utenti, usa `htmlspecialchars`.

```
<p><?= htmlspecialchars($nome, ENT_QUOTES, 'UTF-8') ?></p>
```


E una piccola abitudine che evita molti problemi di sicurezza.

Alternare PHP e HTML senza confusione

Quando lavori in una pagina mista, prova a pensare in due fasi:

1. PHP prepara i dati
2. HTML mostra i dati

```
<?php
$titolo = "Catalogo";
$numeroProdotti = 3;
?>

<h1><?= htmlspecialchars($titolo, ENT_QUOTES, 'UTF-8') ?></h1>
<p>Prodotti disponibili: <?= $numeroProdotti ?></p>
```

Questo ordine evita di infilare calcoli lunghi in mezzo al markup.

Condizioni dentro una pagina

Puoi mostrare HTML diverso in base a una condizione.

```
<?php
$utenteLoggato = false;
?>

<?php if ($utenteLoggato): ?>
    <p>Benvenuto nella tua area personale.</p>
<?php else: ?>
    <p>Accedi per continuare.</p>
<?php endif; ?>
```

Questa forma è leggibile nei template perché separa bene il codice PHP dai pezzi HTML.

Errore comune: fare troppa logica nel template

Se dentro l'HTML inizi ad avere molti calcoli, query al database o controlli lunghi, la pagina diventa difficile da mantenere. Prepara prima i dati in PHP, poi usa il template solo per mostrarli.

Sessioni e cookie

Come conservare informazioni tra una richiesta web e l'altra.

Il problema della memoria nel web

Ogni richiesta web è separata dalle altre. Se apri una pagina e poi un'altra, PHP non ricorda automaticamente chi sei.

Sessioni e cookie servono a conservare informazioni tra una richiesta e la successiva.

Cookie

Un cookie è un piccolo dato salvato nel browser dell'utente.

```
<?php
setcookie("tema", "scuro", time() + 3600);
```

Questo cookie dura un'ora. **3600** sono secondi.

Alla richiesta successiva puoi leggerlo:

```
<?php
$tema = $_COOKIE["tema"] ?? "chiaro";
echo $tema;
```

I cookie sono utili per preferenze semplici, come lingua o tema.

Sessioni

Una sessione salva i dati sul server e collega l'utente a quei dati tramite un identificatore.

Devi chiamare `session_start()` prima di usare `$_SESSION`.

```
<?php
session_start();

$_SESSION["nome"] = "Luca";
```

In un'altra pagina:

```
<?php
session_start();

$nome = $_SESSION["nome"] ?? "ospite";
echo "Ciao $nome";
```

Quando usare le sessioni

Le sessioni sono adatte per login, carrelli e dati temporanei legati alla visita.

I cookie sono adatti per piccole preferenze che possono restare nel browser.

Chiudere una sessione

Per un logout semplice:

```
<?php
session_start();
session_destroy();
```

Attenzione: non salvare password in chiaro dentro cookie o sessioni. Per un login reale servono controlli di sicurezza più completi.

Un esempio di preferenza con cookie

Un cookie è adatto per una preferenza semplice.

```
<?php
setcookie("lingua", "it", time() + 60 * 60 * 24 * 30);
```

Questo cookie dura circa 30 giorni. Alla visita successiva puoi leggere la lingua scelta.

```
<?php
$lingua = $_COOKIE["lingua"] ?? "it";
```

Un esempio di login con sessione

In un login reale dovresti controllare email e password nel database. Qui guardiamo solo il meccanismo della sessione.

```
<?php
session_start();

$_SESSION["utente_id"] = 15;
```

Nelle pagine riservate puoi controllare se l'utente è presente.

```
<?php
session_start();

if (!isset($_SESSION["utente_id"])) {
    echo "Devi accedere";
    exit;
}
```

Errore comune: dimenticare session_start

Se usi `$_SESSION` senza `session_start()`, PHP non apre la sessione. Metti `session_start()` all'inizio del file, prima di stampare HTML.